

Autómatas, Teoría de Lenguajes y Compiladores

Trabajo Práctico

Primera Entrega: Diseño del Lenguaje

04/10/2026

1 Equipo

Nombres	Apellidos	Legajo	E-mail
Gerónimo	Naso Rodríguez	64177	gnasorodriguez@itba.edu.ar
Juan Ignacio	García Vautrin Raggio	63319	jgarciavautrinraggi@itba.edu.ar
Santiago	Fernández Pacheco	65868	sfernandezpacheco@itba.edu.ar
Manuel	Blacker	65184	mblacker@itba.edu.ar

Repositorio: <https://github.com/JuaniRaggio/ccc>

2 Dominio

El proyecto consiste en un compilador que traduce un lenguaje derivado de C a SystemVerilog (IEEE 1800) sintetizable, con énfasis en la detección y explotación del paralelismo inherente al hardware. El objetivo no es solo traducir sintaxis, sino analizar dependencias de datos entre operaciones y generar hardware que aproveche la naturaleza paralela de los circuitos digitales.

El lenguaje de entrada no es un subconjunto estricto de C sino un derivado que toma prestadas algunas construcciones de C++ (como las referencias con `&`) e introduce anotaciones propias (`__parallel`, `unique`, `aliased`) y el mecanismo `constexpr` para evaluación en tiempo de compilación. El foco está en el subconjunto imperativo: no se soporta orientación a objetos ni manejo de excepciones.

El dominio de aplicación es la síntesis de alto nivel (High-Level Synthesis, HLS): el programador describe su algoritmo en un lenguaje imperativo familiar y el compilador genera un módulo de SystemVerilog que puede sintetizarse en hardware real. Esto elimina la necesidad de escribir SystemVerilog a mano, que requiere pensar directamente en términos de ciclos de reloj, registros y señales.

El compilador acepta como entrada un archivo fuente y produce como salida un módulo SystemVerilog sintetizable orientado a FPGAs (Field-Programmable Gate Arrays, circuitos integrados reprogramables en campo) o ASICs (Application-Specific Integrated Circuits, circuitos integrados de propósito específico).

2.1 Alcance del lenguaje de entrada

El compilador soporta las siguientes construcciones:

- Tipos enteros con ancho de bits explícito: `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`
- Tipo booleano: `bool`
- Arreglos unidimensionales de tipos enteros con tamaño estático
- Expresiones aritméticas: `+`, `-`, `*`
- Expresiones lógicas y de comparación: `&&`, `||`, `!`, `==`, `!=`, `<`, `>`, `<=`, `>=`
- Estructuras de control: `if` / `else`, `for`, `while`
- Funciones con parámetros y valor de retorno (cada función se traduce a un módulo SystemVerilog independiente)
- Referencias como parámetros de función con calificadores de aliasing: `unique` (predeterminado) y `aliased`
- Anotación `__parallel` para marcar bloques de sentencias independientes
- Funciones intrínsecas del compilador: `__abs`, `__min`, `__max`
- Variables `constexpr`: constantes evaluadas en tiempo de compilación
- Funciones `constexpr`: funciones evaluadas completamente en tiempo de compilación

No se soportan: memoria dinámica, recursión, tipos de punto flotante, ni llamadas a funciones de la biblioteca estándar de C. La biblioteca estándar no tiene análogo en hardware sintetizable ya que asume la existencia de sistema operativo, heap y descriptores de archivo. Las funciones intrínsecas del compilador reemplazan las operaciones matemáticas más comunes con equivalentes directamente sintetizables.

Las referencias son válidas únicamente como parámetros de función. A diferencia de los punteros de C, no pueden ser nulas ni reasignadas. El concepto de aliasing y los calificadores asociados se definen en la Sección 3.1.2.

3 Construcciones

3.1 Construcciones de entrada

3.1.1 Declaración de función

Cada función de nivel superior se traduce a un módulo SystemVerilog independiente. Los parámetros se convierten en puertos de entrada y el valor de retorno en un puerto de salida.

```
int32_t suma(int32_t a, int32_t b) {  
    return a + b;  
}
```

3.1.2 Referencias y aliasing

El *aliasing* ocurre cuando dos o más nombres distintos refieren al mismo objeto en memoria. En C estándar, dos punteros pueden ser *aliased* (apuntar al mismo objeto) sin que el compilador lo sepa, lo cual impide analizar dependencias de datos y por ende impide paralelizar de forma segura operaciones sobre ellos.

El lenguaje usa referencias en lugar de punteros (tomando la sintaxis de C++) y agrega calificadores explícitos de aliasing. El calificador va entre el `&` y el nombre del parámetro, en línea con cómo C trata `const`. `unique` es el predeterminado e indica que la referencia no se solapa con ninguna otra. `aliased` desactiva esa garantía.

Esta decisión también mejora la seguridad respecto a C estándar: los chequeos de `NULL` generan ramas adicionales que en hardware se traducen en lógica extra y ciclos potencialmente desperdiciados. Con referencias, la garantía de no-nulidad es estática y verificada en tiempo de compilación.

```
// unique es el predeterminado; estas dos firmas son equivalentes
void escalar(int32_t & unique salida, int32_t & unique entrada, int32_t factor);
void escalar(int32_t &salida, int32_t &entrada, int32_t factor);

// aliased: el compilador no puede asumir que salida != entrada
void in_place(int32_t & aliased salida, int32_t &entrada, int32_t factor);
```

El compilador detecta en el sitio de llamada los casos obvios de aliasing (pasar el mismo símbolo dos veces a parámetros `unique`) y emite un error.

3.1.3 Funciones intrínsecas del compilador

Las funciones intrínsecas no pertenecen a ninguna biblioteca: el compilador las traduce directamente a la lógica de hardware correspondiente durante la compilación, sin generar una llamada a función. Esto es análogo a las *primitivas* de otros lenguajes o a las `__builtin_*` de GCC.

Función	Descripción	Hardware generado
<code>__abs(x)</code>	Valor absoluto	Lógica combinacional
<code>__min(x, y)</code>	Mínimo entre dos valores	Comparador + multiplexor
<code>__max(x, y)</code>	Máximo entre dos valores	Comparador + multiplexor

Tabla 1: Funciones intrínsecas del compilador y el hardware que generan.

```
int32_t normalizar(int32_t x, int32_t tope) {
    return __min(__abs(x), tope);
}
```

3.1.4 Bloque `__parallel`

El bloque `__parallel` indica al compilador que las sentencias contenidas no tienen dependencias de datos entre sí y pueden sintetizarse como lógica combinacional independiente dentro del mismo ciclo de reloj.

Es importante distinguir esta semántica de los hilos de software: en hardware, el paralelismo es inherente a la naturaleza de los circuitos. Una señal se propaga simultáneamente por todos los conductores conectados a ella sin coordinación explícita. El bloque `__parallel` no invoca procesos concurrentes; le indica al compilador que ambas operaciones son independientes y pueden por ende pertenecer al mismo ciclo de reloj como bloques `always_comb` separados, en lugar de secuenciarse en una FSM (Finite State Machine, Máquina de Estados Finitos).

```
__parallel {
    resultado_a = f(x);
    resultado_b = g(y);
}
```

3.1.5 Bucle **for** con rango estático

Los bucles **for** cuyos límites son literales o expresiones **constexpr** se sintetizan como FSMs o se desenrollan completamente. Un bucle cuyo límite no sea estático es rechazado, ya que no es posible generar una FSM de tamaño fijo para él.

Una FSM modela la ejecución del bucle como un circuito que avanza de estado en estado con cada flanco de reloj. En cada estado se procesa un elemento del arreglo y se incrementa el índice hasta alcanzar el límite, momento en el que se transiciona al estado final. El *pipeline* de una FSM es la técnica de superponer etapas de distintas iteraciones en ciclos de reloj consecutivos para aumentar el rendimiento.

```
for (int32_t i = 0; i < 8; i++) {
    acum += datos[i];
}
```

3.1.6 Condicional **if / else**

Se traduce a un multiplexor en lógica combinacional (sin estado) o a una transición de estado en una FSM cuando aparece dentro de un bucle secuencial.

```
if (x > umbral) {
    salida = x - umbral;
} else {
    salida = 0;
}
```

3.1.7 Variables y funciones **constexpr**

Las variables **constexpr** son constantes evaluadas en tiempo de compilación. Se traducen a **localparam** en SystemVerilog y pueden usarse en cualquier lugar donde el hardware requiere un valor estático: límites de bucles, tamaños de arreglos, inicializadores.

La semántica es inductiva: una expresión **constexpr** válida es un literal entero (caso base), o una expresión formada exclusivamente por operadores aritméticos y lógicos sobre otras expresiones **constexpr** (paso inductivo). Las funciones **constexpr** extienden esto permitiendo cálculos iterativos en tiempo de compilación.

```
constexpr int32_t N      = 8;
constexpr int32_t MAX_VAL = (1 << N) - 1;
constexpr int32_t TABLA[4] = {1, 2, 4, 8};
```

Las funciones **constexpr** se evalúan completamente en tiempo de compilación y no generan ningún módulo de hardware. Solo pueden recibir y retornar valores **constexpr**, no pueden recibir referencias,

y solo pueden llamar a otras funciones `constexpr`. No pueden ser recursivas: la terminación debe ser garantizable estáticamente mediante un bucle con límite `constexpr`.

```
constexpr int32_t potencia(int32_t base, int32_t exp) {  
    int32_t r = 1;  
    for (int32_t i = 0; i < exp; i++)  
        r *= base;  
    return r;  
}
```

3.2 Construcciones de salida (SystemVerilog)

El lenguaje de salida es SystemVerilog (IEEE 1800), el sucesor de Verilog (IEEE 1364). A diferencia de Verilog, SystemVerilog distingue explícitamente entre lógica combinatorial (`always_comb`) y secuencial (`always_ff`), lo que hace al código generado más legible y permite que las herramientas de síntesis detecten errores de diseño más fácilmente.

3.2.1 Módulo

Cada función del lenguaje de entrada se convierte en un módulo SystemVerilog con puertos `clk`, `rst`, `start`, `done`, los parámetros como entradas y el retorno como salida.

3.2.2 `always_comb`

Para expresiones puramente combinatoriales sin estado: asignaciones directas, operaciones aritméticas, multiplexores generados por condicionales.

3.2.3 `always_ff`

Para lógica secuencial: bucles que requieren múltiples ciclos de reloj, acumuladores, FSMs de control.

3.2.4 Bloques `always_comb` independientes

Cuando el compilador detecta (o el programador declara con `__parallel`) que dos cálculos son independientes, genera dos bloques `always_comb` separados. Dado que ambos son lógica combinatorial, la señal de entrada se propaga simultáneamente a ambos en el mismo ciclo de reloj sin necesidad de coordinación.

4 Casos de Prueba

4.1 Casos de aceptación

El compilador debe aceptar y traducir correctamente los siguientes programas.

N	Descripción	Construcción cubierta
A1	Suma de dos enteros de 32 bits	Expresión aritmética, <code>always_comb</code>
A2	Clasificación de un valor respecto a un umbral	<code>if / else</code> , multiplexor
A3	Suma acumulada de un arreglo de 8 elementos	Bucle <code>for</code> , FSM, <code>always_ff</code>
A4	Cálculo de dos funciones independientes sobre la misma entrada	<code>__parallel</code> , bloques combinacionales
A5	Pipeline: filtrar y luego acumular un arreglo	Composición de módulos, latencia por etapas
A6	Función con condicional dentro de un bucle	Combinación de FSM y multiplexor
A7	Dos referencias <code>unique</code> a variables distintas	Paralelización correcta con <code>unique</code>
A8	Referencia <code>aliased</code> usada correctamente sin paralelismo	<code>aliased</code> deshabilita paralelización
A9	Uso de <code>__abs</code> , <code>__min</code> , <code>__max</code> en expresión	Funciones intrínsecas del compilador
A10	Bucle <code>for</code> con límite literal	Rango estático, FSM o desenrollado
A11	Variable <code>constexpr</code> usada como límite de bucle	<code>constexpr</code> , <code>localparam</code> , rango estático
A12	Función <code>constexpr</code> usada para inicializar un arreglo <code>constexpr</code>	Evaluación en tiempo de compilación

Tabla 2: Casos de prueba de aceptación.

4.2 Casos de rechazo

El compilador debe rechazar los siguientes programas con un mensaje de error descriptivo.

N	Descripción	Razón del rechazo
R1	Función recursiva	No se soporta recursión
R2	Variable de tipo <code>float</code> o <code>double</code>	Tipos de punto flotante no soportados
R3	Llamada a <code>printf</code> , <code>malloc</code> u otra función de <code>stdlib</code>	Funciones de biblioteca estándar no soportadas
R4	Referencia declarada como variable local	Referencias válidas solo como parámetros
R5	Pasar el mismo símbolo dos veces a parámetros <code>unique</code>	Aliasing detectado en sitio de llamada
R6	Bucle <code>for</code> con límite no estático	Rango dinámico no sintetizable como FSM estática
R7	Bloque <code>__parallel</code> con sentencias que escriben la misma variable	Dependencia de datos dentro del bloque
R8	Función sin tipo de retorno explícito	Toda función debe declarar su tipo de retorno
R9	Función <code>constexpr</code> que recibe una referencia	Las funciones <code>constexpr</code> no operan sobre referencias
R10	Función <code>constexpr</code> que llama a una función no <code>constexpr</code>	Solo puede llamar otras funciones <code>constexpr</code>
R11	Función <code>constexpr</code> recursiva	La recursión no garantiza terminación estática

Tabla 3: Casos de prueba de rechazo.

5 Ejemplos

5.1 Operación combinacional con `__parallel`

Dos operaciones independientes sobre la misma entrada se sintetizan como lógica combinacional en el mismo ciclo de reloj. El compilador verifica que `cuadrado` y `absoluto` no comparten escrituras y genera dos bloques `always_comb` separados.

(a) Entrada

```
__parallel {
    cuadrado = x * x;
    absoluto = __abs(x);
}
```

(b) Salida (SystemVerilog)

```
always_comb begin
    cuadrado = x * x;
end

always_comb begin
    if (x < 0)
        absoluto = -x;
    else
        absoluto = x;
end
```

Figura 1: Bloque `__parallel`: entrada en el lenguaje (a) y salida en SystemVerilog (b).

5.2 Condicional como multiplexor

Un `if / else` sin estado se traduce a lógica combinacional con un multiplexor implícito.

(a) Entrada

```
int32_t clamp(int32_t x,
              int32_t tope) {
    if (x > tope)
        return tope;
    else
        return x;
}
```

(b) Salida (SystemVerilog)

```
module clamp (
    input  logic signed [31:0] x,
    input  logic signed [31:0] tope,
    output logic signed [31:0] out
);
    always_comb begin
        if (x > tope)
            out = tope;
        else
            out = x;
        end
    endmodule
```

Figura 2: Condicional `if / else`: entrada en el lenguaje (a) y salida en SystemVerilog (b).

5.3 Bucle con acumulador (FSM)

Un bucle `for` con acumulador requiere retener estado entre ciclos de reloj. El compilador genera una FSM con estados `IDLE`, `COMPUTE` y `DONE`.

(a) Entrada

```
int32_t sumar(int32_t datos[8]) {  
    int32_t acum = 0;  
    for (int32_t i = 0; i < 8; i++)  
        acum += datos[i];  
    return acum;  
}
```

(b) Salida (SystemVerilog)

```
// FSM: IDLE -> COMPUTE -> DONE  
always_ff @(posedge clk) begin  
    case (estado)  
        IDLE: begin  
            acum <= 0; i <= 0;  
            if (start) estado <= COMPUTE;  
        end  
        COMPUTE: begin  
            acum <= acum + datos[i];  
            i <= i + 1;  
            if (i == 7) estado <= DONE;  
        end  
        DONE: begin  
            done <= 1;  
            out <= acum;  
            estado <= IDLE;  
        end  
    endcase  
end
```

Figura 3: Bucle `for` acumulador: entrada en el lenguaje (a) y salida en SystemVerilog (b).

5.4 Hardware parametrizable con `constexpr`

Una función `constexpr` calcula una tabla de potencias de dos en tiempo de compilación. El arreglo `LUT` se convierte en un `localparam` con valores literales y la función `potencia` no genera ningún módulo de hardware.

(a) Entrada

```

constexpr int32_t N = 4;

constexpr int32_t potencia(
    int32_t base,
    int32_t exp
) {
    int32_t r = 1;
    for (int32_t i = 0; i < exp; i++)
        r *= base;
    return r;
}

constexpr int32_t LUT[N] = {
    potencia(2, 0),
    potencia(2, 1),
    potencia(2, 2),
    potencia(2, 3),
};

int32_t buscar(int32_t idx) {
    return LUT[idx];
}

```

(b) Salida (SystemVerilog)

```

// potencia y LUT: resueltos en
// tiempo de compilación, sin
// hardware generado para ellos.

localparam int N = 4;
localparam int LUT [0:3] = {
    1, 2, 4, 8
};

module buscar (
    input  logic signed [31:0] idx,
    output logic signed [31:0] out
);
    always_comb begin
        out = LUT[idx];
    end
endmodule

```

Figura 4: Hardware parametrizable con `constexpr`: entrada en el lenguaje (a) y salida en SystemVerilog (b).

5.5 Rechazo: función `constexpr` recursiva

Las funciones `constexpr` no pueden ser recursivas. La evaluación en tiempo de compilación requiere que el compilador pueda determinar un orden de cómputo finito y estático; la recursión no garantiza terminación sin análisis adicional.

(a) Rechazado

```

// ERROR: constexpr function
// 'factorial' is recursive
constexpr int32_t factorial(
    int32_t n
) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

```

(b) Correcto: iterativo

```

constexpr int32_t factorial(
    int32_t n
) {
    int32_t r = 1;
    for (int32_t i = 2; i <= n; i++)
        r *= i;
    return r;
}

```

Figura 5: Función `constexpr` recursiva: caso rechazado (a) y versión iterativa válida (b).